SASD-GmbH Scientific & Software Development

Architekturdokument

SASD Prompt Manager

Durchdachte Zielarchitektur fuer eine professionelle Anwendung zur Verwaltung, Versionierung, Ausfuehrung, Bewertung und Governance von Prompts

Architekturleitbild
Die Anwendung behandelt Prompts wie langlebige Software- und Wissensartefakte: versioniert, testbar, bewertbar, sicher, portabel und erweiterbar. Die V1 bleibt bewusst klein, waehrend Domain, Schnittstellen und Datenmodell bereits auf Teamarbeit, Workflows, lokale Modelle, Cloud-Modelle und RAG vorbereitet werden.

Feld	Wert
Dokumenttyp	Architekturdokument / Software Architecture Document
Status	Entwurf zur technischen Projektplanung
Version	0.1
Datum	08.05.2026
Bezug	Pflichtenheft und Lastenheft SASD Prompt Manager
Ziel	Konsistente technische Leitarchitektur fuer Entwicklung, Review und spaetere Umsetzung

Hinweis: Das Dokument ist technologisch bewusst nicht dogmatisch, legt aber eine klare empfohlene Zielarchitektur fest. Konkrete Framework-Entscheidungen koennen spaeter durch ADRs finalisiert werden.

Dokumentkontrolle

Version	Datum	Aenderung	Bemerkung
0.1	08.05.2026	Erster konsolidierter Architekturentwurf	Abgeleitet aus Pflichtenheft, Lastenheft und Gespraechsanforderungen

Lesart
Dieses Architekturdokument beschreibt, wie der SASD Prompt Manager technisch strukturiert werden soll. Es ersetzt nicht das Lastenheft und nicht das Pflichtenheft, sondern uebersetzt deren Anforderungen in eine tragfaehige, entwickelbare Softwarearchitektur.

Inhaltsverzeichnis

- 1. Management Summary und Architekturentscheidung
- 2. Zweck, Scope und Beziehung zu Lastenheft/Pflichtenheft
- 3. Architekturtreiber und Qualitaetsziele
- 4. Architekturprinzipien
- 5. Empfohlene Zielarchitektur
- 6. Systemkontext
- 7. Container- und Modularchitektur
- 8. Schichtenmodell und Abhaengigkeitsregeln
- 9. Fachliches Datenmodell und Persistenz
- 10. Prompt-, Template- und Snippet-Engine
- 11. Suche, Indexierung, Kontextmanagement und RAG-Vorbereitung
- 12. Modellintegration, Provider-Adapter und Routing
- 13. Sicherheit, Datenschutz und Governance
- 14. API-, Integrations- und Exportarchitektur
- 15. Benutzeroberflaeche und UX-Architektur
- 16. Workflow-, Test- und Dokumentgenerator-Architektur
- 17. Deployment-, Betriebs- und Backup-Architektur
- 18. Qualitaetssicherung, Observability und Wartbarkeit
- 19. Repository-Struktur und Entwicklungsleitplanken
- 20. Roadmap, Architekturentscheidungen und Risiken
- 21. Abnahmekriterien der Architektur
- Anhang A: Traceability
- Anhang B: Glossar

1. Management Summary und Architekturentscheidung

Kernaussage
Der SASD Prompt Manager sollte als modularer, lokal startbarer und spaeter teamfaehiger modularer Monolith entwickelt werden. Der fachliche Kern bleibt unabhaengig von UI, Datenbank, Suchmaschine und KI-Provider. Alle externen Systeme werden ueber Ports und Adapter angebunden.

Die Architektur verfolgt einen pragmatischen Mittelweg: Die erste Version soll als robuste Einzelanwender- oder kleine Teamloesung realistisch umsetzbar sein. Gleichzeitig darf die Architektur nicht so eng gebaut werden, dass spaetere Funktionen wie Prompt-Tests, semantische Suche, RAG, Modellrouting, Freigabeworkflows, Auditierung oder Teamarbeit nur durch einen Neustart moeglich werden.

Empfohlen wird ein modularer Monolith mit klaren Grenzen zwischen Presentation, API/Application Boundary, Application Core, Domain Model und Infrastructure. Dadurch bleibt die Entwicklung ueberschaubar, waehrend die wichtigsten Komponenten spaeter in eigene Services ausgelagert werden koennen, falls die Produktreife dies rechtfertigt.

Architekturentscheidung	Empfehlung	Begruendung
Grundstil	Modularer Monolith mit Ports-and-Adapters	Minimiert V1-Komplexitaet, haelt Domain sauber und erlaubt spaetere Auslagerung.
Betriebsmodell V1	Local-first oder kleiner Intranet-Betrieb	Datenschutzfreundlich, offlinefaehig und fuer Einzelanwender sofort nutzbar.
Datenhaltung V1	Relationale lokale Datenbank plus offene Exporte	SQLite eignet sich fuer V1; PostgreSQL/MariaDB bleiben als spaetere Teamoption moeglich.
KI-Anbindung	Adapter fuer Cloud- und lokale Modelle	Kein Provider-Lock-in; Governance kann Modellwahl steuern.
Suche V1	Volltextsuche plus strukturierte Filter	Geringer Aufwand, hoher Nutzen; semantische Suche wird vorbereitet.
Sicherheit	Datenklassifizierung, RBAC, Secret-Warnungen, Audit-Logs	Sicherheitsfunktionen muessen frueh im Datenmodell verankert sein.

2. Zweck, Scope und Beziehung zu Lastenheft/Pflichtenheft

2.1 Zweck des Architekturdokuments

Dieses Dokument beschreibt die technische Zielarchitektur fuer den SASD Prompt Manager. Es beantwortet insbesondere, aus welchen Bausteinen das System bestehen soll, wie diese Bausteine zusammenarbeiten, wo Verantwortlichkeiten liegen, welche technischen Risiken zu beachten sind und welche Architekturentscheidungen fuer V1 verbindlich oder zumindest empfohlen sind.

2.2 Beziehung zu Lastenheft und Pflichtenheft

Dokument	Rolle	Beziehung zur Architektur
Lastenheft	Beschreibt den Bedarf des Auftraggebers und den fachlichen Nutzen.	Die Architektur muss zeigen, wie dieser Bedarf technisch tragfaehig abgedeckt wird.
Pflichtenheft	Konkretisiert die fachlichen und technischen Anforderungen.	Die Architektur ordnet Anforderungen Komponenten, Schnittstellen und

Dokument	Rolle	Beziehung zur Architektur
		Datenobjekten zu.
Architekturdokument	Beschreibt Struktur, Prinzipien, technische Entscheidungen und Integrationsmuster.	Es bildet die Bruecke zwischen Anforderungen und Implementierung.

2.3 In-Scope der Architektur

- Fachlicher Kern fuer Prompts, Versionen, Snippets, Templates, Projekte, Ergebnisse, Bewertungen und Richtlinien.
- UI-, API-, Storage-, Search-, Export-, Governance- und LLM-Adapter als klar getrennte Bausteine.
- V1-faehige Architektur fuer lokale Nutzung und spaetere Teamfaehigkeit.
- Vorbereitung fuer semantische Suche, RAG, Prompt-Tests, Workflows, Dokumentgeneratoren und Modellrouting.
- Sicherheits- und Datenschutzarchitektur inklusive Datenklassifizierung, Rollenmodell und Auditierung.

2.4 Out-of-Scope dieser Architekturversion

- Endgueltige Auswahl aller konkreten Frameworks und UI-Bibliotheken.
- Produktive SaaS-Clusterarchitektur mit Mandantenabrechnung und Hochverfuegbarkeit.
- Vollstaendige IDE-Plugin-Architektur fuer mehrere Entwicklungsumgebungen.
- Rechtlich zertifizierte Compliance- oder Audit-Plattform.
- Autonome Agenten mit Schreibrechten auf Produktivsysteme ohne menschliche Kontrolle.

3. Architekturtreiber und Qualitaetsziele

Architekturtreiber sind die Anforderungen, die die Struktur des Systems besonders stark beeinflussen. Beim SASD Prompt Manager sind dies vor allem Datenschutz, Portabilitaet, Versionierung, Erweiterbarkeit, Nachvollziehbarkeit und eine bewusst klein gehaltene V1.

Treiber	Auswirkung auf die Architektur	Konsequenz
Prompts als Artefakte	Prompts benoetigen Identitaet, Lebenszyklus, Historie und Metadaten.	Eigene Domain-Entitaeten statt reiner Textdateiablage.
Versionierung und Diff	Aenderungen muessen nachvollziehbar und rueckrollbar sein.	Append-only PromptVersion-Ansatz mit Vergleichsdiensten.
Projektbezug und Checklisten	Prompts muessen in Projektkontexten auffindbar und nutzbar sein.	Project-Aggregat mit Prompt-Zuordnungen und UI-Listenansicht.
Datenschutz und Geheimnisse	Sensible Daten duerfen nicht unkontrolliert an Modelle gehen.	Datenklassifizierung, Policy Engine und Provider-Regeln im Core.
Providerneutralitaet	Cloud- und lokale Modelle sollen austauschbar sein.	Model-Port mit Adapterimplementierungen je Provider.
Offene Formate	Export und Backup duerfen nicht proprietaer sein.	Markdown/YAML/JSON/ZIP als erste Architekturanforderung.
Spaetere RAG-Funktion	Kontextquellen und Embeddings sollen spaeter moeglich sein.	KnowledgeSource und Search-Port frueh im Datenmodell vorsehen.
Kleine robuste V1	Projekt darf sich nicht in Enterprise-Komplexitaet verlieren.	Modularer Monolith statt Microservices.

3.1 Priorisierte Qualitaetsziele

Prioritaet	Qualitaetsziel	Beschreibung
1	Wartbarkeit	Aenderungen an UI, Storage oder Provider duerfen die Domain nicht zerstoeren.
2	Nachvollziehbarkeit	Prompt-Versionen, Ausfuehrungen, Bewertungen und Richtlinienentscheidungen muessen auditierbar sein.
3	Datenschutzfaehigkeit	Sensible Inhalte muessen erkannt, klassifiziert und providerbezogen gesteuert werden koennen.
4	Portabilitaet	Nutzer muessen ihre Prompt-Bibliothek in offenen Formaten sichern und migrieren koennen.
5	Erweiterbarkeit	Suche, Modelladapter, Workflows und RAG muessen erweiterbar bleiben.
6	Benutzbarkeit	V1 muss ohne Expertenwissen als Prompt-Bibliothek nuetzlich sein.

4. Architekturprinzipien

ID	Prinzip	Architekturregel
P-01	Domain first	Prompt, Version, Projekt, Snippet, Ausfuehrung und Richtlinie werden als fachliche Konzepte modelliert, nicht als zufaellige Tabellen oder UI-Formulare.
P-02	Ports and Adapters	Datenbank, Suchmaschine, LLM-Provider, Exportformate und UI werden ueber Schnittstellen angebunden.
P-03	Local-first by default	Die V1 muss ohne zwingende Cloud funktionieren. Cloud-Modelle sind optional, nicht Voraussetzung.
P-04	Security by design	Datenklassifizierung, Secret-Erkennung, Rollen und Audit werden nicht nachtraeglich angeflanscht.
P-05	Open data	Daten sollen aus dem Produkt heraus wiedergewinnbar bleiben: Markdown, YAML, JSON und ZIP sind Pflichtpfade.
P-06	Small core, extensible edges	Der Kern bleibt klein; fortgeschrittene Funktionen werden als Module/Adapter ergänzt.
P-07	Explainable governance	Wenn eine Ausfuehrung blockiert oder gewarnt wird, muss das System den Grund nennen.
P-08	Human in control	Automatische Optimierung und Agentenfunktionen duerfen produktive Prompts nicht ohne menschliche Freigabe veraendern.

5. Empfohlene Zielarchitektur

5.1 Grundform: Modularer Monolith

Die empfohlene Architektur ist ein modularer Monolith. Alle Kernmodule werden in einem deploybaren System betrieben, besitzen aber intern klare Grenzen. Dadurch entsteht kein frueher Microservice-Overhead, aber dennoch eine saubere Struktur fuer spaetere Erweiterungen.

Warum kein Microservice-Start?
Microservices loesen vor allem Skalierungs- und Organisationsprobleme. In V1 stehen aber Produktklarheit, Datenmodell, UX und Sicherheit im Vordergrund. Ein modularer Monolith ist einfacher zu testen, zu installieren, zu sichern und weiterzuentwickeln.

Architekturvariante	Bewertung	Empfehlung
Reine Desktop-Anwendung	Sehr gut fuer Datenschutz und Offline-Nutzung, aber spaetere Teamarbeit schwieriger.	Als Packaging-Option interessant, nicht als einziges Architekturmodell.
Reine Web-SaaS-Plattform	Teamfaehig, aber fuer V1 zu teuer, komplex und datenschutzrechtlich schwerer.	Nicht fuer V1.
Lokale Web-App mit Backend	Guter Kompromiss aus UI-Flexibilitaet, API-Faehigkeit und lokaler Nutzung.	Empfohlene V1-Richtung.
Modularer Monolith mit API und Adaptern	Sauberer Kern, spaetere Auslagerung moeglich.	Zielarchitektur.

5.2 Empfohlener Technologiepfad

Die Architektur ist nicht zwingend an eine Sprache gebunden. Fuer die konkrete SASD-Umsetzung bietet sich jedoch ein Stack an, der lokal gut laeuft, testbar ist und spaeter API- und Teamfunktionen unterstuetzt.

Baustein	Empfohlene Richtung	Alternative / Hinweis
Backend / Application Core	.NET 8 / ASP.NET Core oder vergleichbar sauber geschichteter Backend-Stack	Alternativ PHP/Node.js, wenn Webhosting bei IONOS zwingend wird. Domainmodell sollte portabel bleiben.
Frontend	React/TypeScript oder Blazor; klare Komponenten fuer Bibliothek, Projektliste, Editor und Dialoge	WPF/Avalonia moeglich, wenn Desktop-first entschieden wird.
Datenbank V1	SQLite fuer lokale Einzelanwenderinstallation	PostgreSQL/MariaDB fuer Team- oder Serverbetrieb vorbereiten.
Suche V1	Relationale Volltextsuche oder eingebetteter Suchindex	Spaeter Embeddings/Vektorindex.
LLM-Integration	Provider-Adapter mit Modellprofilen	Cloud- und lokale Provider gleich behandeln.
Export	Markdown/YAML, JSON, ZIP	Git-freundliche Ablage optional.

6. Systemkontext

Im Systemkontext steht der SASD Prompt Manager zwischen Anwendern, Projektwissen, lokalen und cloudbasierten KI-Modellen sowie offenen Export- und Backupformaten. Er ist nicht selbst das KI-Modell, sondern steuert, dokumentiert und sichert die Prompt-Nutzung.

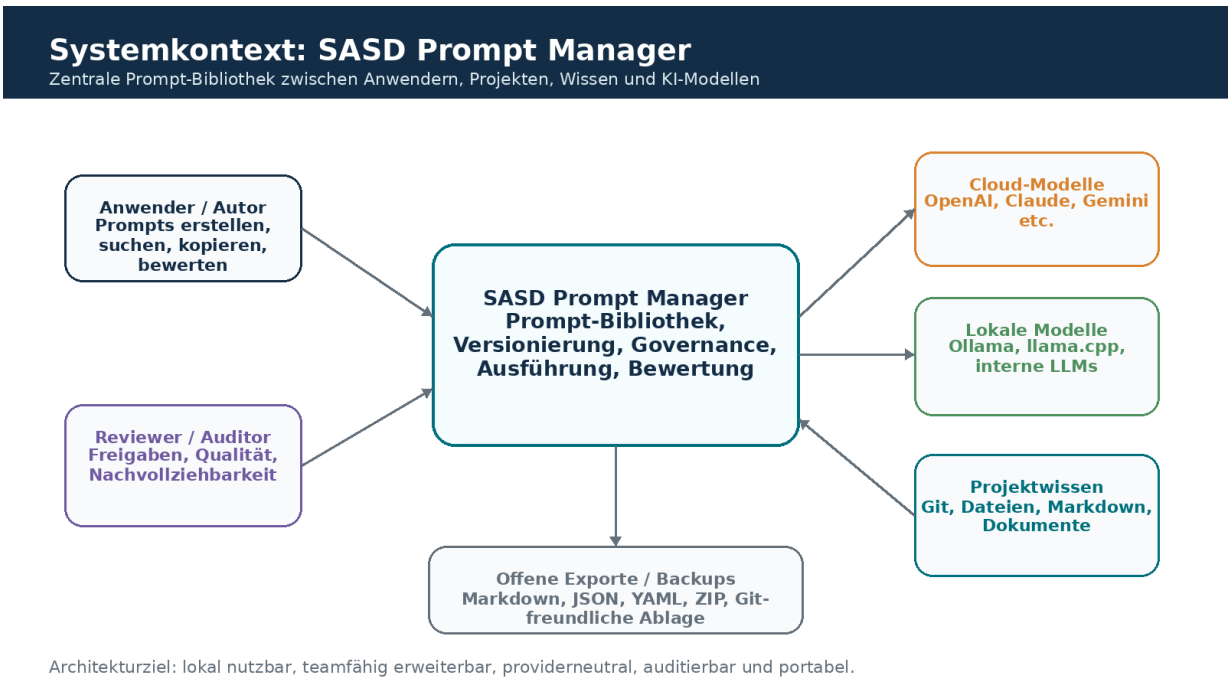

Abbildung 1: Systemkontext des SASD Prompt Managers.

6.1 Externe Akteure und Systeme

Akteur/System	Interaktion	Architekturimplikation
Anwender / Autor	Erstellt, strukturiert, kopiert, testet und verbessert Prompts.	UI muss schnell, klar und bibliotheksorientiert sein.
Reviewer / Maintainer	Prueft Versionen, gibt Prompts frei, pflegt Bausteine und Profile.	Statusmodell, Kommentare, Review- und Rollenrechte.
Auditor / Datenschutzrolle	Prueft Ausfuehrungen, Richtlinienverstoesse und Datenklassifizierung.	Audit-Log, unveraenderbare Protokolle, Export fuer Pruefungen.
Cloud-LLM-Provider	Fuehrt Prompts aus, falls erlaubt und konfiguriert.	Provider-Adapter, API-Key-Verwaltung, Kosten- und Modellparameter.
Lokale LLMs	Fuehren sensible Prompts lokal aus.	Adapter fuer lokale Endpunkte und Offlinebetrieb.
Projektwissen	Liefert Kontext aus Dateien, Dokumenten, Repositories und Zusammenfassungen.	KnowledgeSource-Modell, Rechtepruefung und Kontext-Vorschau.

7. Container- und Modularchitektur

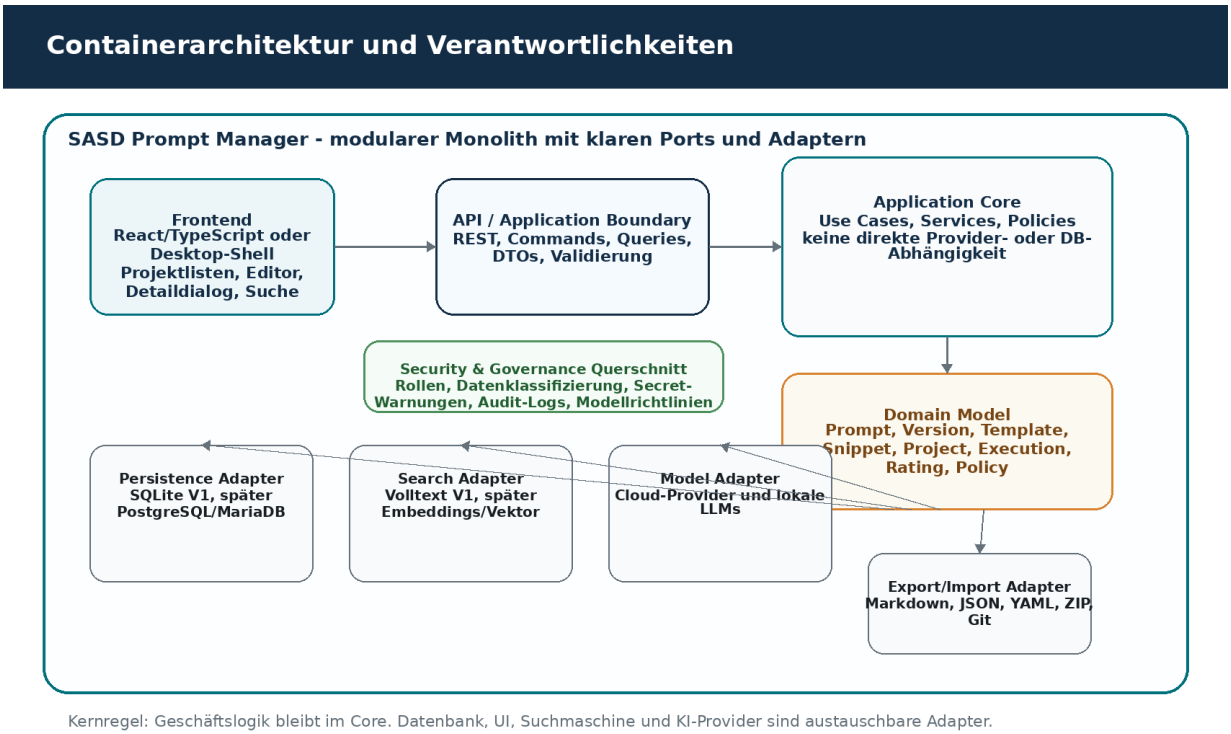

Abbildung 2: Container- und Modularchitektur.

7.1 Hauptmodule

Modul	Verantwortung	Darf nicht enthalten
Frontend	Darstellung, Navigation, Editor, Projektlisten, Detaildialoge, Suchmasken, Bewertungsoberflaeche.	Keine Datenbankzugriffe, keine Provider-Logik, keine Governance-Entscheidungen.
API/Application Boundary	Entgegennahme von Requests, Auth-Kontext, DTO-Validierung, Fehlerkontrakte.	Keine komplexe Domänenlogik, keine direkten SQL-Details.
Application Core	Use Cases, Workflows, Policy-Auswertung, Versionierungslogik, Transaktionsgrenzen.	Keine UI-Details, keine Provider-SDKs.
Domain Model	Entitaeten, Wertobjekte, Statusregeln, Invarianten und fachliche Validierung.	Keine Infrastruktur, kein HTTP, keine Dateisystemzugriffe.
Infrastructure	Repository-Implementierungen, Datenbank, Suchindex, Dateisystem, LLM-Adapter, Exportadapter.	Keine fachlichen Sonderentscheidungen ausserhalb definierter Ports.
Security & Governance	Rollen, Berechtigungen, Secret-Erkennung, Datenklassifizierung, Modellrichtlinien, Audit.	Keine UI-Abhaengigkeiten, keine provider-spezifische Sonderlogik.

8. Schichtenmodell und Abhaengigkeitsregeln

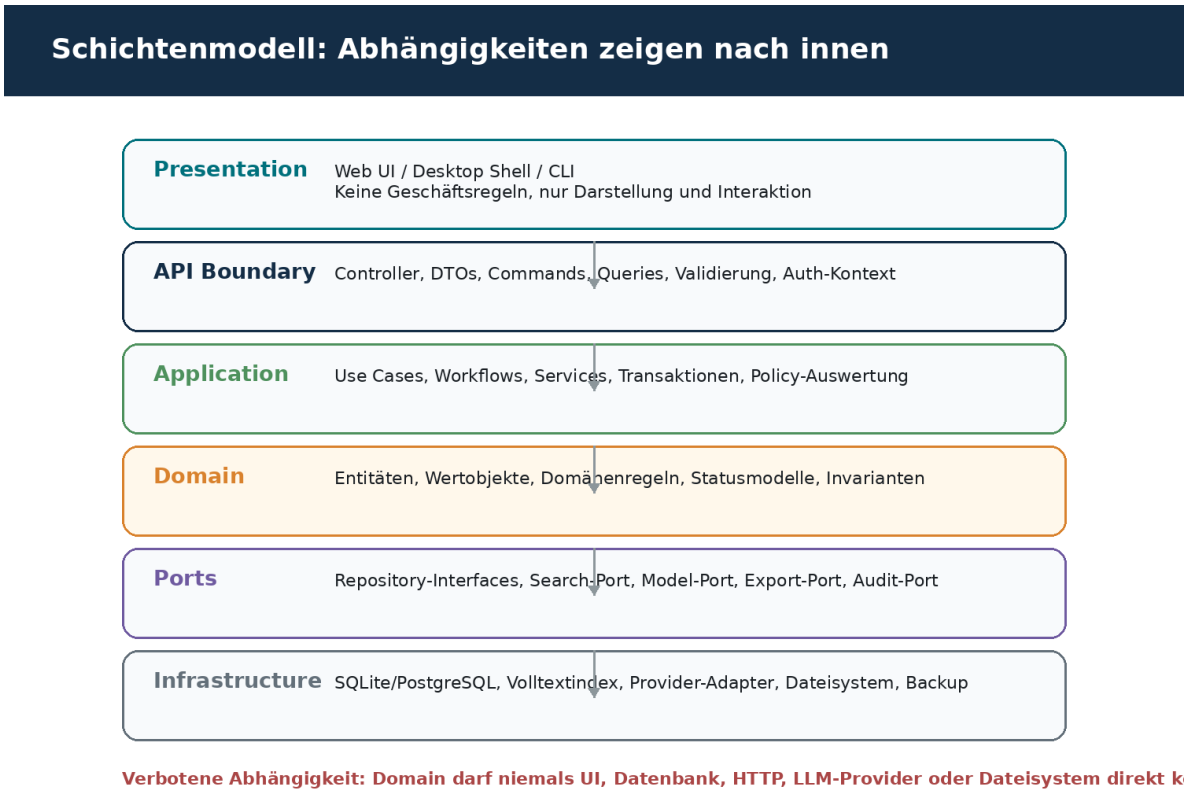

Abbildung 3: Schichtenmodell und Abhaengigkeitsrichtung.

8.1 Abhaengigkeitsregeln

Regel	Beschreibung	Pruefung
R-01	Domain kennt keine Infrastruktur.	Kein Import/Using von HTTP-, DB-, Dateisystem- oder Providerbibliotheken in Domain-Projekten.
R-02	Application orchestriert, Domain entscheidet.	Use Cases rufen Domain-Methoden auf, statt fachliche Regeln in Controller zu schreiben.
R-03	Adapter implementieren Ports.	Provider-, Such- und Exportadapter werden nur ueber Interfaces angesprochen.
R-04	DTOs bleiben an der Grenze.	API-DTOs werden nicht als Domainobjekte missbraucht.
R-05	Policies sind zentrale Regeln.	Modellverbote, Datenklassifizierung und Freigaben werden nicht in UI-Ifs verteilt.
R-06	Aenderungen sind testbar.	Jede Kernregel erhaelt Unit- oder Integrationstests.

9. Fachliches Datenmodell und Persistenz

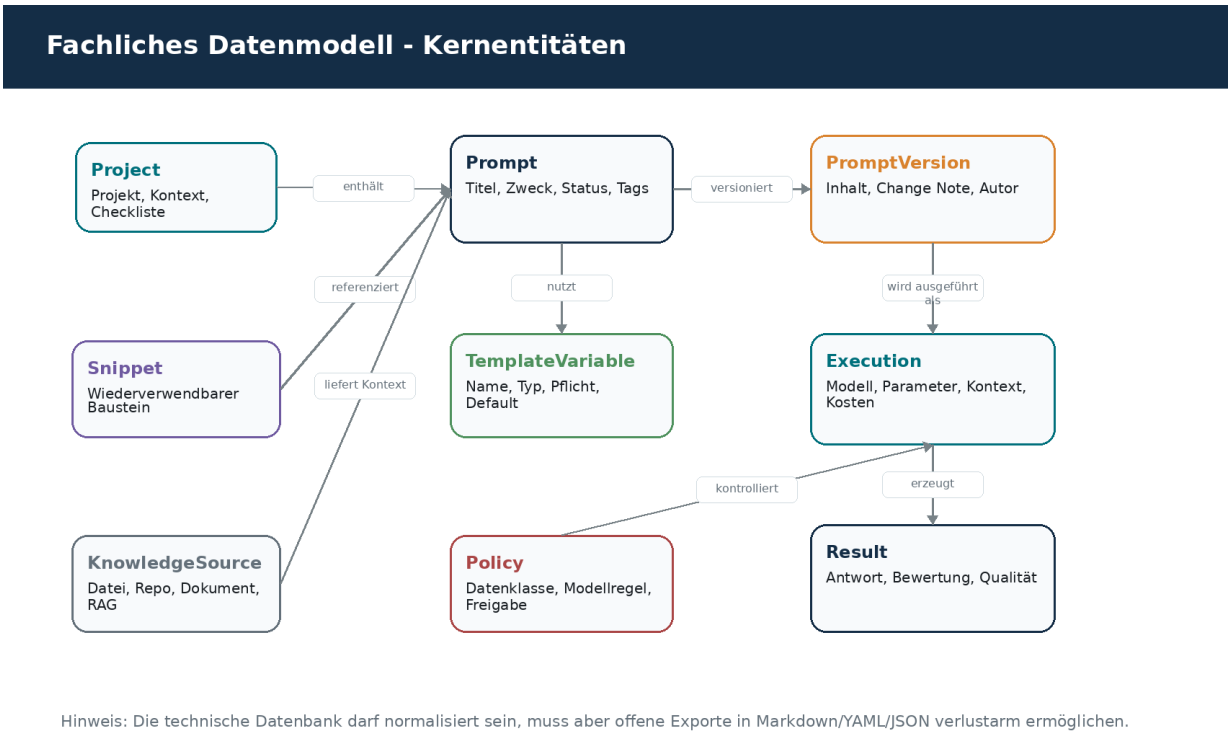

Abbildung 4: Fachliches Kernmodell.

9.1 Kernaggregate

Aggregat	Zweck	Wichtige Felder / Beziehungen
Project	Buendelt Prompts, Kontextquellen, Checklisten und Workflows.	id, name, description, status, owner, tags, promptAssignments, knowledgeSources
Prompt	Fachliche Identitaet eines Prompts.	id, title, summary, category, tags, status, language, dataClass, currentVersionId
PromptVersion	Unveraenderlicher Stand eines Promptinhalts.	id, promptId, versionNo, content, changeNote, authorId, createdAt, derivedFromVersionId
Snippet	Wiederverwendbarer Prompt-Baustein.	id, name, content, status, dataClass, version, usages
TemplateVariable	Beschreibt Platzhalter und Eingabemasken.	name, type, required, defaultValue, options, validationRule, helpText
Execution	Dokumentiert eine Ausfuehrung oder Kopiervorbereitung.	promptVersionId, modelProfileId, parameters, contextRefs, tokenCount, cost, policyDecision
Result	Speichert Antwort, Bewertung und Nacharbeit.	executionId, response, rating, comments, qualityFlags, acceptedAsGoodExample
Policy	Formuliert Governance-Regeln.	scope, condition, action, severity, explanation, reviewerRequired

9.2 Persistenzstrategie

Die Persistenz soll zwei Ziele gleichzeitig erreichen: eine konsistente transaktionale Arbeitsdatenbank fuer die Anwendung und offene, menschenlesbare Exporte fuer Backup, Migration und Git-nahe Nutzung. Daher sollte die technische Datenbank nicht das einzige Format sein, in dem Prompts ueberleben koennen.

Ebene	V1-Ansatz	Spaetere Erweiterung
Operative Datenbank	SQLite lokal oder relationale DB im Serverbetrieb.	PostgreSQL/MariaDB mit Migrationen und ggf. Mandantenfaehigkeit.
Dateibasierte Exporte	Markdown-Dateien mit YAML-Header und separatem JSON-Manifest.	Git-Synchronisation und Review ueber Pull Requests.
Auditdaten	Anhaengende Audit-Events mit Zeitstempel und Akteur.	Manipulationsschutz, Hashketten oder revisionssichere Exportformate.
Suchindex	Aus DB rekonstruierbar, nicht alleinige Quelle.	Embedding- und Vektorindex als abgeleitete Daten.
Kontextquellen	Referenzen und Metadaten, keine unkontrollierte Kopie sensibler Daten.	RAG-Pipeline mit Chunking, Embeddings und Rechtepruefung.

```
---
id: prompt-php-security-review
version: 4
status: approved
tags: [php, security, rest, sasd]
data_class: internal
model_policy: local-or-approved-cloud
---
# Prompt
Analysiere den folgenden {{LANGUAGE}}-Code nach {{STANDARD}} ...
```

10. Prompt-, Template- und Snippet-Engine

Die Prompt-Engine ist der fachliche Kern fuer das Zusammenbauen eines finalen Prompts. Sie darf nicht nur Text ersetzen, sondern muss auch Variablen, Snippets, Basis-Prompts, Datenklassifizierung, Vorschau und Richtlinienpruefung beruecksichtigen.

Funktion	Architekturverhalten	V1/V2
Template-Variablen	Parser erkennt {{VARIABLE}}, ordnet Typen und Pflichtfelder zu und erzeugt Eingabemasken.	V1
Final Prompt Preview	Vor Kopieren oder Ausfuehrung wird exakt angezeigt, welcher Text entsteht.	V1
Snippet-Einbindung	Bausteine werden referenziert und kontrolliert expandiert.	V1/V2
Basis-Prompts / Vererbung	Prompts koennen gemeinsame Regeln oder Rollenprofile erben.	V2
Abhaengigkeitsanalyse	Aenderungen an Snippets zeigen betroffene Prompts.	V2
Semantische Zerlegung	Lange Prompts werden in Ziel, Kontext, Regeln, Beispiele und Ausgabeformat segmentiert.	V3

Architekturregel fuer Prompt-Rendering

Das System muss zwischen gespeicherter Vorlage, expandiertem Prompt, tatsaechlich versendetem Prompt und

Ergebnis unterscheiden. Nur so sind Versionierung, Datenschutzpruefung und Reproduktion sauber moeglich.

11. Suche, Indexierung, Kontextmanagement und RAG-Vorbereitung

11.1 Suche in V1

V1 benoetigt eine schnelle, nachvollziehbare Suche nach Titel, Beschreibung, Inhalt, Tags, Kategorie, Projekt, Status, Sprache und Autor. Diese Suche darf einfach beginnen, muss aber bereits so gekapselt werden, dass spaeter eine semantische Suche ergaenzt werden kann.

Suchfunktion	V1	Spaeter
Volltextsuche	Index ueber Promptitel, Zusammenfassung, Inhalt und Tags.	Ranking, Highlighting, Synonyme.
Strukturfilter	Kategorie, Projekt, Tag, Status, Autor, Sprache, Datenklasse.	Gespeicherte Filter und Suchprofile.
Aehnlichkeit	Noch optional oder einfache Heuristik.	Embeddings und Vektorindex.
Duplikaterkennung	Textaehnlichkeit und Titel/Tag-Hinweise.	Semantische Duplikatanalyse.
Kontextauswahl	Manuelle Zuordnung von Wissensquellen.	Automatisches Retrieval mit Vorschau und Richtlinienpruefung.

11.2 RAG-Vorbereitung

RAG sollte nicht Bestandteil der kleinsten V1 sein, aber die Datenmodelle muessen vorbereitet werden. Eine KnowledgeSource darf nicht nur ein Dateipfad sein, sondern braucht Herkunft, Datenklasse, Aktualitaet, Berechtigungen, Extraktionsstatus und Beziehung zu Projekten oder Prompts.

- KnowledgeSource speichert Quelle, Typ, Datenklasse, Besitzer, erlaubte Nutzungsarten und Aktualisierungsstatus.
- KnowledgeChunk ist spaeter eine abgeleitete Einheit fuer Chunking, Embedding und semantische Suche.
- Vor jeder automatischen Kontextbeigabe muss eine Vorschau sichtbar sein.
- Governance-Regeln muessen verhindern koennen, dass sensible Quellen an ungeeignete Modelle gehen.
- Der Suchindex bleibt rekonstruktionsfaehig; die Datenbank und Exporte bleiben fuehrend.

12. Modellintegration, Provider-Adapter und Routing

Der SASD Prompt Manager soll Cloud-Modelle und lokale Modelle unter einer einheitlichen Abstraktion behandeln. Die Anwendung darf nicht hart an ein einzelnes KI-Modell gekoppelt werden.

Konzept	Beschreibung	Architekturanforderung
ModelProfile	Beschreibt Provider, Modellname, Temperatur, Max Tokens, Kostenmodell, Datenschutzstufe und erlaubte Nutzung.	Versionierbares Konfigurationsobjekt.
ModelProviderPort	Einheitliche Schnittstelle zum Ausfuehren von Prompts.	Adapter je Provider.
RoutingPolicy	Waehlt oder verbietet Modelle nach Aufgabe, Datenklasse, Kosten, Verfuegbarkeit und Nutzerrolle.	Policy Engine im Application Core.
CostTracker	Speichert Tokenverbrauch, Laufzeit,	Teil von Execution/Result.

Konzept	Beschreibung	Architekturanforderung
	Kosten und Fehlerraten.	
LocalModelAdapter	Unterstuetzt lokale Endpunkte, z. B. Ollama oder llama.cpp.	Kein Cloudzwang.
Dry-run / Copy-only	Erlaubt Nutzung ohne API-Anbindung durch Kopieren des finalen Prompts.	V1-MUSS-Funktion.

V1-Empfehlung
Die direkte Modellanbindung kann in V1 hinter dem Kopier-Workflow zurueckstehen. Wichtig ist aber, dass das Datenmodell bereits Execution, ModelProfile und PolicyDecision kennt. So muss die Architektur spaeter nicht umgebaut werden.

13. Sicherheit, Datenschutz und Governance

Sicherheit ist beim Prompt Manager keine Randfunktion. Prompts koennen Kundendaten, Projektinterna, Quellcode, API-Keys, personenbezogene Daten oder strategische Unternehmensinformationen enthalten. Deshalb muessen Security und Governance frueh und sichtbar in die Architektur eingebaut werden.

Sicherheitsbereich	V1-Mindestumfang	Spaetere Haertung
Authentisierung	Lokales Benutzerprofil oder einfache Login-Struktur bei Serverbetrieb.	SSO, MFA, zentrale Identitaetsanbieter.
Autorisierung	Rollen Leser, Autor, Reviewer, Maintainer, Admin, Auditor.	Feingranulare Rechte pro Projekt, Prompt, Datenklasse.
Datenklassifizierung	Public, Internal, Confidential, Restricted.	Eigene Klassifizierungsprofile je Organisation.
Secret-Erkennung	Warnung bei API-Keys, Passwoertern, Tokens, Private Keys.	Erweiterbare Pattern, ML-Unterstuetzung, Quarantaene.
Modellrichtlinien	Cloud-Modelle fuer Restricted blockieren oder Review verlangen.	Mandantenfaehige Policy Engine.
Audit-Logs	Aenderungen, Ausfuehrungen, Freigaben und Warnungen protokollieren.	Manipulationsschutz und Export fuer Audits.
Datenschutz	Loeschung, Export, Anonymisierung personenbezogener Inhalte vorbereiten.	DSGVO-Workflows, Aufbewahrungsfristen, DPA-Verwaltung.

13.1 Policy-Entscheidungsmodell

Jede Ausfuehrung oder jeder Export kann eine Policy-Entscheidung erhalten: erlaubt, erlaubt mit Warnung, erfordert Review oder blockiert. Diese Entscheidung soll eine kurze maschinenlesbare und menschenlesbare Begruendung enthalten.

```
PolicyDecision {
  decision: "allowed_with_warning",
  reason_code: "CONFIDENTIAL_TO_CLOUD_MODEL",
  message: "Der Prompt enthaelt vertrauliche Inhalte. Cloud-Ausfuehrung ist nur nach Bestaetigung erlaubt.",
  reviewer_required: false,
  logged: true
}
```

14. API-, Integrations- und Exportarchitektur

Auch wenn V1 lokal starten kann, sollte die Anwendung intern API-faehig entworfen werden. Das erleichtert spaetere Web-UIs, Desktop-Shells, CLI-Tools, IDE-Plugins und Automatisierung.

API-Bereich	Beispiele	Architekturhinweis
Prompts	/api/prompts, /api/prompts/{id}, /api/prompts/{id}/versions	CRUD plus Versionierung getrennt behandeln.
Projects	/api/projects, /api/projects/{id}/prompts	Projektlisten und Checklistenansicht als eigene Use Cases.
Templates	/api/templates/render, /api/snippets	Rendering immer mit Vorschau und Policy Check.
Search	/api/search/prompts	Search-Port verwenden, nicht direkt Datenbank-SQL im Controller.
Executions	/api/executions, /api/executions/{id}/result	Ausfuehrung, Ergebnis und Kosten getrennt speichern.
Governance	/api/policies, /api/audit	Nur fuer berechtigte Rollen sichtbar.
Export/Import	/api/export, /api/import	Offene Formate und Validierungsberichte.

14.1 Exportarchitektur

- Markdown/YAML: menschenlesbarer Export einzelner Prompts und Promptpakete.
- JSON: maschinenlesbarer Export fuer Migration, API-Nutzung und Tests.
- ZIP: Gesamtbackup mit Manifest, Prompts, Snippets, Projekten, Ergebnissen und optionalen Anhängen.
- Git-freundlich: stabile Dateinamen, kleine Dateien, deterministische Sortierung.
- Importvalidierung: jeder Import erzeugt einen Bericht ueber Konflikte, Duplikate, fehlende Snippets und Versionskollisionen.

15. Benutzeroberflaeche und UX-Architektur

Die UI muss die einfache taegliche Nutzung priorisieren: suchen, oeffnen, kopieren, anpassen, speichern, versionieren. Expertenfunktionen duerfen nicht die V1-Oberflaeche ueberfrachten.

UI-Bereich	V1-Funktion	Architekturbaustein
Start / Dashboard	Favoriten, zuletzt verwendet, Projektzugriff, schnelle Suche.	Frontend Feature Module: Dashboard
Prompt-Bibliothek	Liste, Filter, Tags, Kategorie, Status.	Feature Module: PromptLibrary
Projektansicht	Checklisten-/Listenansicht mit Promptttiteln.	Feature Module: ProjectWorkspace
Prompt-Detaildialog	Vollstaendiger Prompt, Kopieren, Editieren, Versionen, Bewertung.	Reusable Dialog/Panel Component
Editor	Mehrzeiliger Text, Markdown-Grundstruktur, Variablenliste, Vorschau.	Editor Component + Template Service
Versionen	Historie, Diff, Rollback, Change Note.	Version Feature Module
Governance	Warnungen, Datenklasse, Modellentscheidung, Audit-Hinweise.	Security UI Components

UI-Grundsatz

Der normale Anwender soll nicht zuerst eine KI-Orchestrierungsplattform sehen, sondern eine sehr gute Prompt-Bibliothek. Expertenmodus, Workflow-Engine, Modellrouting und RAG werden sichtbar, wenn sie gebraucht

werden.

16. Workflow-, Test- und Dokumentgenerator-Architektur

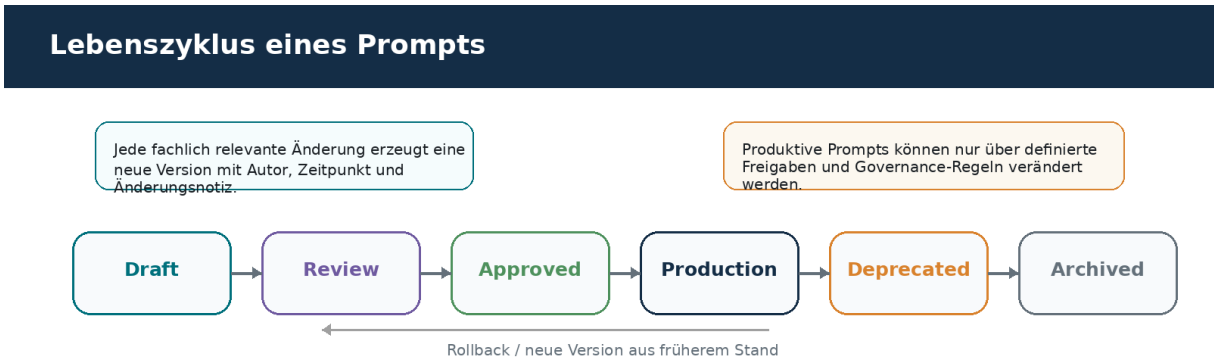

Abbildung 5: Lebenszyklus eines Prompts.

16.1 Prompt-Workflow-Engine

Workflows sollen in V1 noch nicht als vollstaendige BPMN-Engine gebaut werden. Dennoch sollte die Architektur einen einfachen Workflow-Mechanismus vorbereiten: geordnete Schritte, Eingaben, erzeugte Artefakte, Freigaben und gespeicherte Zwischenergebnisse.

Workflowtyp	Beispiel	Architekturhinweis
Prompt-Optimierung	Prompt V3 aus Antwortbewertung verbessern.	Versionierung und Bewertungsdaten verbinden.
SASD-Dokumentation	Lastenheft -> Pflichtenheft -> Architektur -> ADR -> Security Review.	WorkflowDefinition mit StepTemplates.
Security Review	Code/Architektur pruefen, Findings erzeugen, Härtungsbacklog ableiten.	Ergebnisse strukturiert speichern, nicht nur Text.
Prompt-Testlauf	Testfaelle gegen Prompt-Version ausfuehren und Ergebnis bewerten.	TestCase, ExpectedSignal, EvaluationResult.

16.2 Testarchitektur fuer Prompts

- PromptTestCase speichert Eingabe, erwartete Signale, unerwuenschte Ausgaben und Mindestbewertung.
- PromptRegressionRun fuehrt mehrere Testfaelle gegen eine Prompt-Version und ein Modellprofil aus.
- EvaluationResult dokumentiert, ob der Prompt weiterhin akzeptabel arbeitet.
- Manuelle Bewertung bleibt zunaechst massgeblich; automatische Bewertung kann spaeter ergaenzt werden.
- Produktive Prompts duerfen nur nach bestandenen Mindesttests oder dokumentierter Ausnahme freigegeben werden.

17. Deployment-, Betriebs- und Backup-Architektur

Betriebsmodus	Beschreibung	Geeignet fuer	Architekturhinweis
Lokaler Einzelplatz	Anwendung und lokale Datenbank laufen auf einem Rechner.	V1, Datenschutz, schnelle Nutzung.	Backup/Export besonders wichtig.
Lokaler Server / Intranet	Backend laeuft auf internem Server, mehrere Clients greifen zu.	SASD-intern, kleines Team.	Rollen, Audit und zentrale Backups aktivieren.
Desktop-Shell mit lokalem Backend	Desktop-App startet oder nutzt lokalen Dienst.	Benutzerfreundliche lokale Installation.	UI bleibt austauschbar, Core bleibt gleich.
Spaetere Web-/SaaS-Variante	Mehrbenutzerbetrieb mit Mandanten und hoher Verfuegbarkeit.	Produktreife, externe Kunden.	Erst nach stabiler V1/V2 sinnvoll.

17.1 Backup und Restore

Backup und Restore sind keine Zusatzfunktion, sondern Produktgrundlage. Der Wert der Anwendung liegt in der Prompt-Bibliothek und ihren historischen Verbesserungen. Datenverlust waere fatal.

- Manueller Gesamtexport als ZIP mit Manifest.
- Automatischer periodischer lokaler Backupvorschlag.
- Restore-Testfunktion mit Validierung, bevor bestehende Daten ueberschrieben werden.
- Export der Prompt-Bibliothek ohne Ergebnisse, falls Antworten sensible Daten enthalten.
- Optionaler verschluesselter Backupcontainer fuer vertrauliche Prompt-Sammlungen.

18. Qualitaetssicherung, Observability und Wartbarkeit

Bereich	Architekturmassnahme	Nutzen
Unit-Tests	Domain- und Application-Services ohne Datenbank und UI testen.	Schnelle Sicherheit bei Refactorings.
Integrationstests	Repository-, Export-, Import-, Such- und Provideradapter testen.	Schnittstellenfehler frueh erkennen.
UI-Smoke-Tests	Kernablaeufe: Prompt anlegen, suchen, oeffnen, kopieren, versionieren.	V1 bleibt nutzbar.
Security Tests	Rollen, Datenklassifizierung, Secret-Erkennung und Policy-Entscheidungen pruefen.	Datenschutz- und Governance-Regeln bleiben wirksam.
Observability	Strukturierte Logs, Audit-Events, Fehlerkontrakte.	Fehleranalyse ohne Preisgabe sensibler Inhalte.
Architekturtests	Pruefung verbotener Abhaengigkeiten zwischen Projekten/Modulen.	Schichtenmodell bleibt sauber.

Logging-Hinweis
Logs duerfen keine vollstaendigen Prompts oder Modellantworten enthalten, sofern diese vertrauliche Inhalte haben koennen. Stattdessen sollten IDs, Hashes, Datenklassen, Ereignistypen und technische Fehlerdetails protokolliert werden.

19. Repository-Struktur und Entwicklungsleitplanken

Die Repository-Struktur sollte die Architektur sichtbar machen. Fachliche Module, Tests und Dokumentation gehoeren von Beginn an sauber getrennt, damit das Projekt nicht als UI-Prototyp ohne tragfaehigen Kern endet.

```
sasd-prompt-manager/
├── docs/
│   ├── lastenheft/
│   ├── pflichtenheft/
│   ├── architecture/
│   ├── adr/
│   └── security/
├── src/
│   ├── Sasd.PromptManager.Domain/
│   ├── Sasd.PromptManager.Application/
│   ├── Sasd.PromptManager.Api/
│   ├── Sasd.PromptManager.Infrastructure/
│   ├── Sasd.PromptManager.Security/
│   └── Sasd.PromptManager.Export/
├── frontend/
│   └── sasd-prompt-manager-ui/
├── tests/
│   ├── Domain.Tests/
│   ├── Application.Tests/
│   ├── Infrastructure.Tests/
│   └── Api.IntegrationTests/
├── samples/
│   └── demo-prompt-library/
└── tools/
    ├── import-export/
    └── smoke-tests/
```

19.1 Entwicklungsleitplanken

- Vor Implementierung jeder groesseren Funktion wird geprueft, welchem Modul sie gehoert.
- Domain- und Application-Projekte erhalten keine UI- oder Provider-Abhaengigkeiten.
- Jede neue Schnittstelle wird mit Beispielrequest, Beispielresponse und Fehlerfaellen dokumentiert.
- Jede Datenmodellmigration wird nachvollziehbar versioniert.
- Jede Funktion mit Datenschutzbezug erhaelt mindestens einen Security-Testfall.
- Architekturentscheidungen werden als ADR dokumentiert.

20. Roadmap, Architekturentscheidungen und Risiken

20.1 Architekturreife nach Releases

Release	Architekturfokus	Enthaltene Kernbausteine
V1	Robuste Prompt-Bibliothek	Prompt, Project, Version, Tags, Editor, Variablen, Suche, Kopieren, Export, Grundrechte, Datenklasse.
V1.1	Qualitaet und Nachvollziehbarkeit	Diff, Rollback, Bewertungen, Audit-Basics, Importvalidierung, bessere Backups.

Release	Architekturfokus	Enthaltene Kernbausteine
V2	Ausfuehrung und Governance	ModelProfile, Provideradapter, Ergebnisverwaltung, Kosten, Policy Engine, Reviewstatus.
V2.5	Workflows und Tests	Prompt-Testfaelle, Regression, Dokumentationsworkflows, Snippet-Abhaengigkeiten.
V3	Wissenssystem und Teamplattform	Semantische Suche, RAG, Rollen pro Projekt, Freigaben, lokale/Cloud-Routingregeln, ggf. Marketplace.

20.2 Vorgeschlagene ADRs

ADR	Titel	Entscheidungsvorschlag
ADR-001	Architekturstil	Modularer Monolith mit Ports-and-Adapters.
ADR-002	Betriebsmodell V1	Local-first mit spaeterer Teamoption.
ADR-003	Datenhaltung V1	SQLite/Relationale DB plus offene Exporte.
ADR-004	Prompt-Versionierung	Append-only Versionen mit Rollback als neue Version.
ADR-005	Modellintegration	Provideradapter und ModelProfile statt harter Providerbindung.
ADR-006	Datenklassifizierung	Datenklasse als Pflichtkonzept fuer Prompts, Ergebnisse und Quellen.
ADR-007	RAG-Vorbereitung	KnowledgeSource im Datenmodell, Vektorindex erst spaeter.
ADR-008	UI-Komplexitaet	Einfachmodus in V1, Expertenmodus schrittweise.

20.3 Architekturrisiken

Risiko	Auswirkung	Gegenmassnahme
V1 wird zu gross	Projekt verzettelt sich vor produktiver Nutzbarkeit.	V1 strikt auf Bibliothek, Versionierung, Suche, Projektansicht, Export und Grundsicherheit begrenzen.
UI treibt Datenmodell	Schnelle Oberflaeche erzeugt spaeter unwartbaren Kern.	Domainmodell zuerst, UI ueber Use Cases anbinden.
Datenschutz zu spaet	Cloud-Ausfuehrung sensibler Inhalte ohne Kontrolle.	Datenklassifizierung und PolicyDecision ab V1.
Provider-Lock-in	Kosten, Verfuegbarkeit und Datenschutz haengen an einem Anbieter.	ModelProviderPort und Adapter.
Export als Nachgedanke	Daten werden im System gefangen.	Offene Formate als V1-Abnahmekriterium.
RAG voreilig	Komplexitaet steigt, bevor Grundnutzen steht.	RAG vorbereiten, aber erst nach stabiler Bibliothek bauen.

21. Abnahmekriterien der Architektur

Die Architektur gilt fuer den Start der Implementierung als tragfaehig, wenn die folgenden Kriterien erfuellt und in Code, Dokumentation oder Tests nachweisbar sind.

ID	Kriterium	Nachweis
AK-A01	Domain, Application, API/UI und Infrastructure sind getrennte Module.	Repository-Struktur und Build-Abhaengigkeiten.
AK-A02	Prompt, PromptVersion, Project, Snippet, TemplateVariable, Execution, Result und Policy sind als fachliche Konzepte modelliert.	Domainmodell und Datenbankschema.
AK-A03	Prompt-Rendering erzeugt eine Vorschau und unterscheidet Vorlage, expandierten Prompt und Ausfuehrung.	Unit- und Integrationstests.
AK-A04	Aenderungen an Prompts erzeugen Versionen und koennen verglichen oder zurueckgerollt werden.	Use-Case-Test.
AK-A05	Datenklassifizierung und erste Secret-Warnungen sind eingebaut.	Security-Testfall.
AK-A06	Export in offene Formate ist verlustarm moeglich.	Export-/Import-Smoke-Test.
AK-A07	LLM-Anbindung erfolgt ueber Adapter oder ist sauber vorbereitet.	ModelProviderPort / Interface.
AK-A08	Suchfunktion ist gekapselt und kann spaeter ersetzt oder erweitert werden.	SearchPort und Implementierung.
AK-A09	Audit-relevante Ereignisse werden strukturiert erfasst.	Audit-Service und Testdaten.
AK-A10	V1 bleibt ohne Cloudmodell nutzbar.	Copy-only Workflow laeuft offline.

Anhang A: Traceability

Anforderungsbereich aus Pflichtenheft	Architekturbaustein	Bemerkung
Prompt-Bibliothek, Organisation, Metadaten	Domain: Prompt, Project; UI: PromptLibrary; Persistence Repository	V1-Kern.
Editor, Templates, Variablen	Prompt Rendering Service, TemplateVariable, Editor Component	V1 mit Grundfunktionen.
Snippets, Module, Basis-Prompts	Snippet Aggregate, Dependency Analyzer	Grundmodell V1, Analyse V2.
Versionierung, Varianten, Lebenszyklus	PromptVersion, Statusmodell, Diff Service	V1/V1.1.
Suche und Wissensorganisation	SearchPort, SearchIndex, KnowledgeSource	V1 Volltext, V2/V3 semantisch.
Kontextmanagement und RAG	KnowledgeSource, ContextPreview, spaeter KnowledgeChunk	V1 vorbereiten, spaeter ausbauen.
Ausfuehrung und Modellrouting	ModelProfile, ModelProviderPort, RoutingPolicy, Execution	V2, Copy-only bereits V1.
Ergebnisverwaltung und Bewertung	Result, Rating, QualityFlags	V1.1/V2.
Prompt-Linter und Tests	PromptLinterService, PromptTestCase, RegressionRun	V2/V2.5.
Workflows und Dokumentgenerator	WorkflowDefinition, WorkflowRun, StepResult	V2.5/V3.
Sicherheit und Governance	Policy Engine, RBAC, DataClassification, AuditLog	Ab V1.
Export/Import und Backup	ExportAdapter, ImportValidator, BackupManifest	Ab V1.

Anhang B: Glossar

Begriff	Bedeutung
Prompt	Anweisung, Kontext oder Aufgabenstellung an ein KI-Modell.
Prompt-Version	Unveraenderlicher Stand eines Promptinhalts inklusive Metadaten.
Snippet	Wiederverwendbarer Prompt-Baustein.
Template	Prompt-Vorlage mit Variablen und optionalen Snippets.
ModelProfile	Konfiguration fuer ein KI-Modell inklusive Parameter, Kosten und Governance.
PolicyDecision	Ergebnis einer Governance-Pruefung: erlaubt, Warnung, Review erforderlich oder blockiert.
RAG	Retrieval Augmented Generation; gezielte Einbindung relevanter Wissensquellen.
Port	Abstrakte Schnittstelle des Anwendungskerns zu externer Infrastruktur.
Adapter	Konkrete Implementierung eines Ports, z. B. Datenbank, Provider oder Exportformat.
Modularer Monolith	Eine deploybare Anwendung mit intern klar getrennten Modulen.

Zusammenfassung

Architekturempfehlung

Der SASD Prompt Manager sollte mit einer kleinen, stabilen V1 starten: Prompt-Bibliothek, Projektlisten, Detaildialog, Editor, Variablen, Versionierung, Suche, Export, Datenklassifizierung und Grundrechte. Alle fortgeschrittenen Funktionen werden architektonisch vorbereitet, aber nicht in die erste Umsetzung gepresst.

Die vorgeschlagene Architektur schafft eine solide Grundlage fuer die spaetere Entwicklung zu einer Prompt-Engineering-Plattform. Entscheidend ist, den fachlichen Kern sauber zu halten und jede neue Funktion in klaren Modulen zu verorten. Damit bleibt das System wartbar, datenschutzfaehig und langfristig produktfaehig.